ASSIGNMENT

OPERATING SYSTEM WITH LINUX

Name: Abhishek Vincent
Roll No: 03
Batch: MCA-26
Submitted to: Mrs. Shoby Sunny

1. Banker's Algorithm - Deadlock Avoidance.

	Allocation A B C D	Max A B C D
T_0	3 1 4 1	6 4 7 3
T_1	2 1 0 2	4 2 3 2
T_2	2 4 1 3	2 5 3 3
T_3	4 1 1 0	6 3 3 2
T_4	2 2 2 1	5 6 7 5

Total number of instances of each resource type = A - 15, B - 11, C - 10, D - 11

a) Need Matrix

Need = Max - Allocation

T_0 : {6, 4, 7, 3} - {3, 1, 4, 1} = {3, 3, 3, 2}

T_1 : {4, 2, 3, 2} - {2, 1, 0, 2} = {2, 1, 3, 0}

T_2 : {2, 5, 3, 3} - {2, 4, 1, 3} = {0, 1, 2, 0}

T_3 : {6, 3, 3, 2} - {4, 1, 1, 0} = {2, 2, 2, 2}

T_4 : {5, 6, 7, 5} - {2, 2, 2, 1} = {3, 4, 5, 4}

b) Available vector = Total Resources - current allocation matrix

current allocation matrix

A = 3 + 2 + 2 + 4 + 2 = 13

B = 1 + 1 + 4 + 1 + 2 = 9

C = 4 + 0 + 1 + 1 + 2 = 8

D = 1 + 2 + 3 + 0 + 1 = 7

Total Allocated = {13, 9, 8, 7}

Available = {15, 11, 10, 11} − {13, 9, 8, 7}

= {2, 2, 2, 4}

c) Applying Banker's Algorithm.

Available = {2, 2, 2, 4}

Work = {2, 2, 2, 4}

Check $need_i \leq work$

1. Check T_0

{3, 3, 3, 2} ≤ {2, 2, 2, 4} => No (cannot execute)

2. Check T_1

{2, 1, 3, 0} ≤ {2, 2, 2, 4} → No (cannot execute)

3. Check T_2

{0, 1, 2, 0} ≤ {2, 2, 2, 4} → Yes

work = work + allocation

= {2, 2, 2, 4} + {2, 4, 1, 3}

= {4, 6, 3, 7}

4. Check T_3

{2, 2, 2, 2} ≤ {4, 6, 3, 7}

work = work + allocation

= {4, 6, 3, 7} + {4, 1, 1, 0} = {8, 7, 4, 7}

5. Check T_4

$\{3, 4, 5, 4\} \leq \{8, 7, 4, 7\}$ → No (cannot execute)

6. Check T_0

$\{3, 3, 3, 2\} \leq \{8, 7, 4, 7\}$ → Yes

work = work + allocation

$= \{8, 7, 4, 7\} + \{3, 1, 4, 1\}$

$= \{11, 8, 8, 8\}$

7. Check T_1

$\{2, 1, 3, 0\} \leq \{11, 8, 8, 8\}$

work = work + allocation

$= \{11, 8, 8, 8\} + \{2, 1, 0, 2\} = \{13, 9, 8, 10\}$

8 Check T_4 $\{3, 4, 5, 4\} \leq \{13, 9, 8, 10\}$

work = work + allocation

$= \{13, 9, 8, 10\} + \{2, 2, 2, 1\} = \{15, 11, 10, 11\}$

Safe sequence = $T_2 \rightarrow T_3 \rightarrow T_0 \rightarrow T_1 \rightarrow T_4$

d) If T_4 request = $\{2, 2, 2, 4\}$

Need = $\{5, 6, 7, 5\} - \{2, 2, 2, 1\} = \{3, 4, 5, 4\}$

1. Check Request $\leq$ Need $_{T_4}$

$\{2, 2, 2, 4\} \leq \{3, 4, 5, 4\}$ → True

2. Check Request $\leq$ Available

$\{2,2,2,4\} \leq \{2,2,2,4\} \rightarrow$ True

Available $= \{2,2,2,4\} - \{2,2,2,4\} = \{0,0,0,0\}$

Allocation$_{T_4} = \{2,2,2,1\} + \{2,2,2,4\} = \{4,4,4,5\}$

Need$_{T_4} = \{3,4,5,4\} - \{2,2,2,4\} = \{1,2,3,0\}$

Conclusion:-

No process can be executed because the new remaining available resources are zero. Therefore the system will enter an unsafe state.

2. Disk Scheduling Algorithms

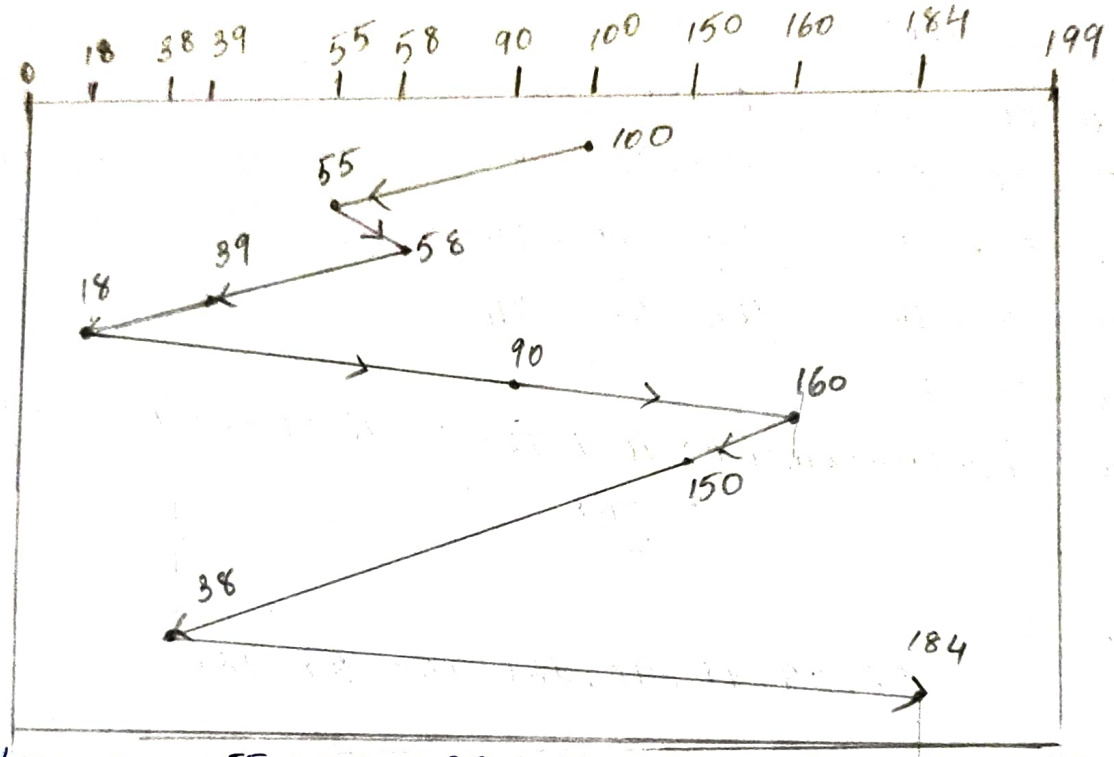
Given

Total Tracks : 200 (0 to 199)

Initial Head position : 100

Disk Requests : 55, 58, 39, 18, 90, 160, 150, 38, 184

i FCFS (First-come First Serve)



Order: 100 → 55 → 58 → 39 → 18 → 90 → 160 → 150 → 38 → 184

Head movements

55 - 100 = 45 | 160 - 90 = 70

58 - 55 = 3 | 150 - 160 = 10

39 - 58 = 19 | 38 - 150 = 112

18 - 39 = 21 | 184 - 38 = 146

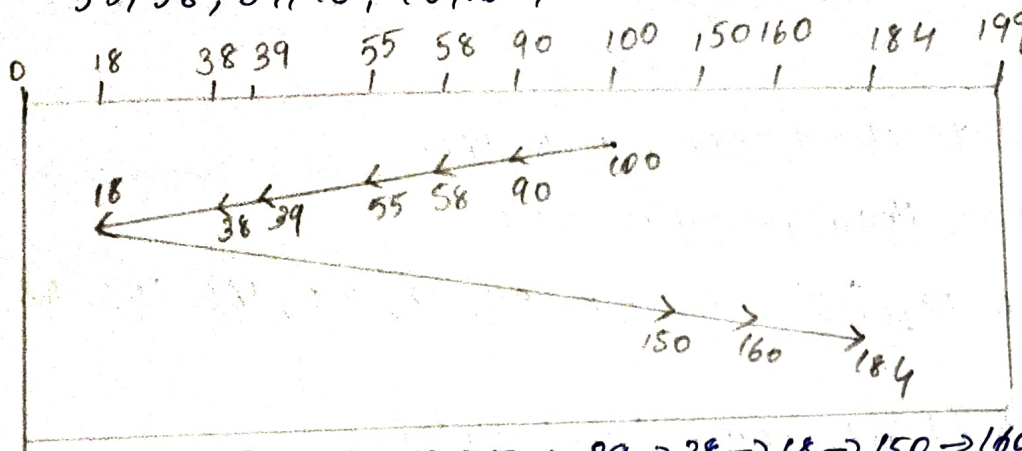
90 - 18 = 72

Total Head movements = 45 + 3 + 19 + 21 + 72 + 70 + 10 + 112 + 146

= 498

ii) Shortest Seek Time First (SSTF)

55, 58, 39, 18, 90, 160, 150, 38, 184



Order:- 100 → 90 → 58 → 55 → 39 → 38 → 18 → 150 → 160 → 184

Head movements:

90 - 100 = 10

58 - 90 = 32

55 - 58 = 3

39 - 55 = 16

38 - 39 = 1

18 - 38 = 20

150 - 18 = 132

160 - 150 = 10

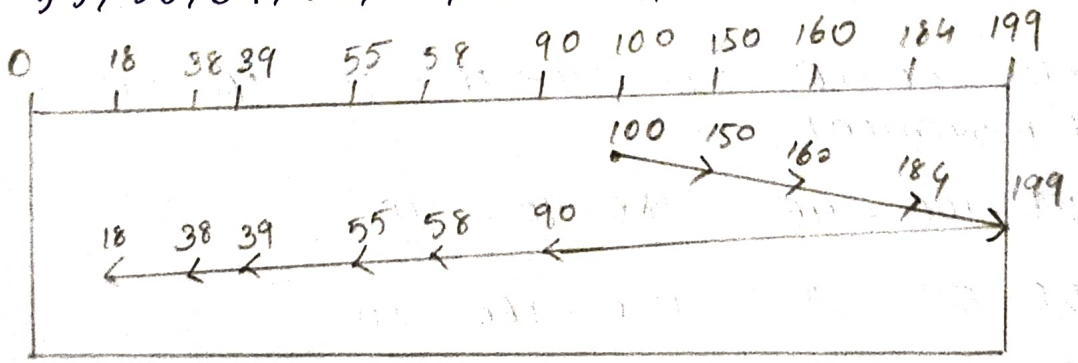
184 - 160 = 24

Total head movements = 10 + 32 + 3 + 16 + 1 + 20 + 132 + 10 + 24

= 248

iii) SCAN

55, 58, 39, 18, 90, 160, 150, 38, 184



Order: 100 → 150 → 160 → 184 → 199 → 90 → 58 → 55 → 39 → 38 → 18

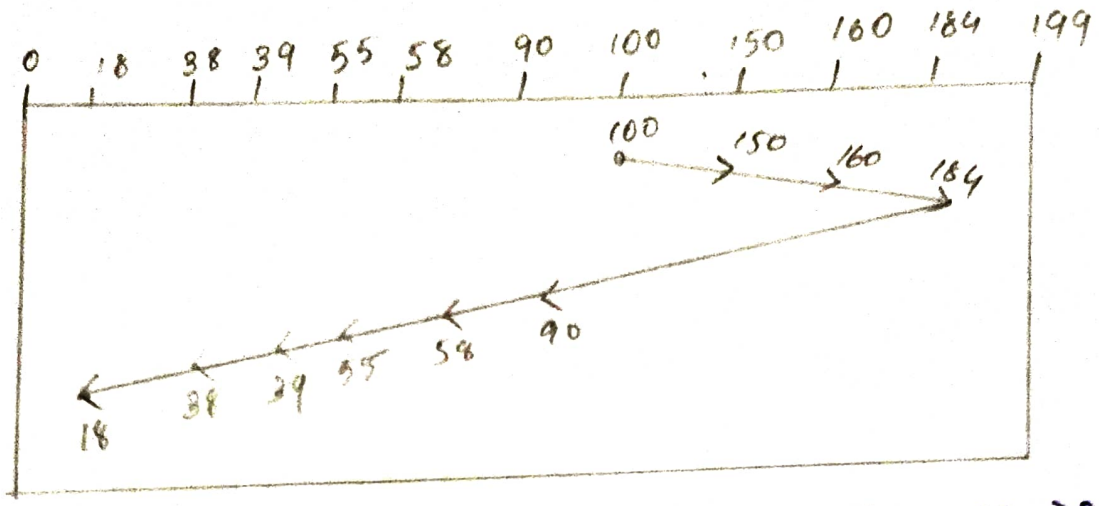
Head movements:

199 - 100 = 99

18 - 199 = 181

Total Head movements = 99 + 181 = 280

iv) LOOK



Order : 100 → 150 → 160 → 184 → 90 → 58 → 55 → 39 → 38 → 18

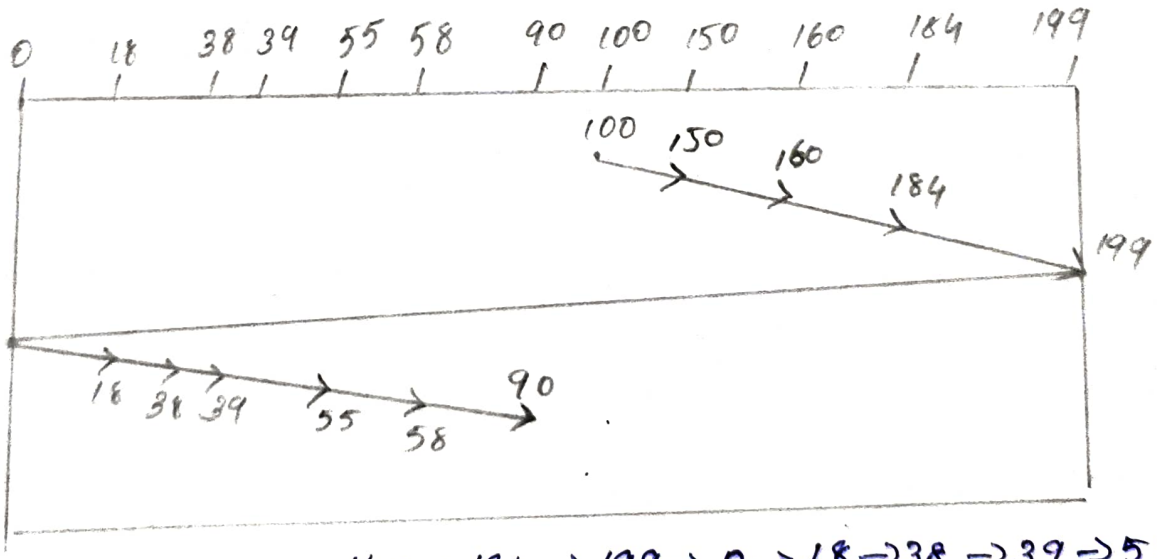
Head movements

184 − 100 = 84

18 − 184 = 166

Total head movements 84 + 166 = 250

v) C-SCAN



Order: 100 → 150 → 160 → 184 → 199 → 0 → 18 → 38 → 39 → 55 → 58 → 90

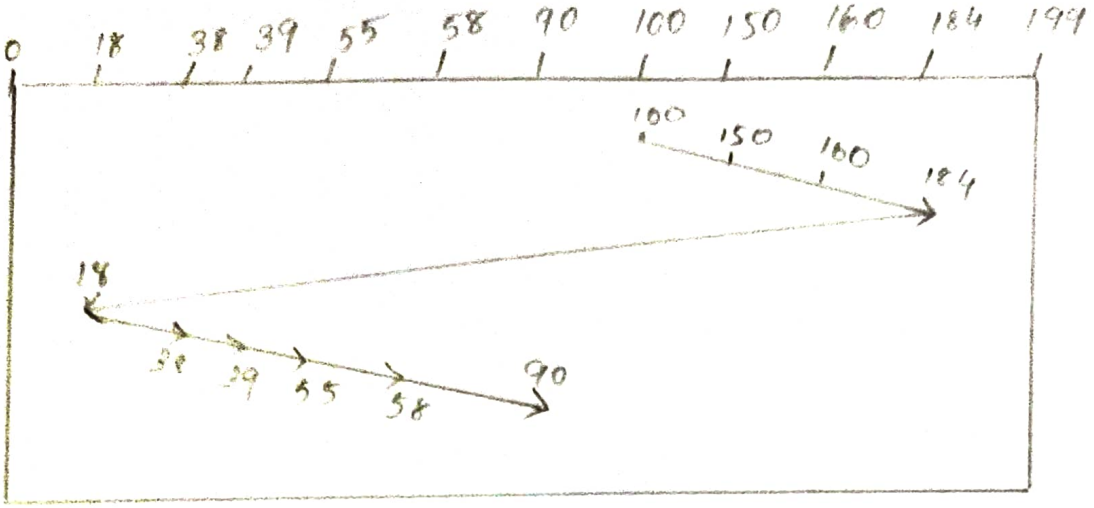
199 − 100 = 99

0 − 199 = 199

90 − 0 = 90

Total head movements = 388 tracks

vi) C-LOOK



Order:- 100 → 150 → 160 → 184 → 18 → 38 → 39 → 55 → 58 → 90

199 - 100 = 84

18 - 184 = 166

90 - 18 = 72

Total head movements = 84 + 166 + 72 = 322

3. Deadlock Recovery

a) Methods used for recovering from a deadlock

1. Process Termination:
   Terminating one or more deadlocked processes to break the circular wait dependency loop.
2. Resource preemption:
   Successively reclaiming allocated resources from processes and allocating them to other processes until the deadlock cycle is broken.

b) Process Termination

1. Abort all deadlocked processes:
   This is a drastic but simple method. The OS terminates every process in the deadlock loop. While it guarantees deadlock resolution, it incurs a high cost because all partial computations completed by these processes are lost.
2. Abort one process at a time.
   The OS terminates one process in the cycle and reruns the deadlock detection algorithm to see if the cycle is broken. The process repeats until the deadlock is resolved. This minimizes discarded computational work but introduces significant CPU overhead by running algorithm multiple times.

Factors to consider when selecting a victim process

1. Process Priority :- Low-priority processes may be terminated first.
2. Computation Time: How long the process has computed so far, and how much longer it needs to finish.
3. Resources consumed: The types and total count of resources the process currently holds.
4. Resources needed: How many more resources the process requires to finish its execution path.
5. Process Nature : Consider whether the process is interactive, a system process or a batch process

c) Resource Preemption

Resource preemption is a deadlock recovery method in which the operating system temporarily takes resources away from a process and gives them to another process to break the deadlock.

It involves 3 important issues

1. Victim Selection

The operating system selects a process from which resources can be taken. The selection should minimize the total cost of recovery.

Factors include process priority, resources held, amount of work completed, and the cost of restarting the process.

2. Rollback

When a resource is taken away, the affected process may not be able to continue safely. The operating system may roll back the process to a previous safe state and restart it later.

Rollback can be:

- Total Rollback: Restart the process from the beginning.
- Partial Rollback: Return the process to a previously saved checkpoint.

3. Starvation.

Starvation occurs when the same process is repeatedly selected as the victim and never gets enough resources to complete its execution. It can be prevented by considering the number of times a process has been selected as a victim. For example, the operating system can avoid repeatedly preempting resources from the same process.

d) Comparison between process termination and resource preemption.

Feature	Process Termination	Resource Preemption
Meaning	Terminates one or more deadlocked processes	Takes resources away from selected processes

Resource Release	Resources are released when process terminate	Resources are released through preemption.
Complexity	Relatively simple to implement	More comple to implement.
Starvation	Can occur if the same process is repeatedly terminated and restarted	Can occur if the same process is repeatedly selected as a victim.
Advantages	Simple and effective; terminating all deadlocked processes quickly removes the deadlock	Allows process to continue later without necessarily terminating them permanently.
Disadvantages	May lose significant work and disrupt users	Requires careful victim selection, rollback support and starvation prevention.
Recovery cost	Can be high if important processes are terminated	Can be high due to rollback and restarting.